

EXPERIMENT 11

Map Colouring to Implement CSP

Aim

To write a Python program to implement Map Colouring using Constraint Satisfaction Problem (CSP).

Algorithm

1. Define the regions and their neighboring regions.
2. Define a set of colors as the domain for each region.
3. Select an uncolored region.
4. Assign a color that does not conflict with its neighboring regions.
5. If no valid color is available, backtrack and try another color.
6. Repeat until all regions are colored.
7. Display the final map coloring.

Program

```
# Map Coloring using CSP
```

```
colors = ["Red", "Green", "Blue"]
```

```
graph = {
```

```
    "A": ["B", "C"],
```

```
    "B": ["A", "C", "D"],
```

```
"C": ["A", "B", "D"],  
"D": ["B", "C"]  
}
```

```
coloring = {}
```

```
def is_safe(region, color):
```

```
    for neighbor in graph[region]:
```

```
        if neighbor in coloring and coloring[neighbor] == color:
```

```
            return False
```

```
    return True
```

```
def solve():
```

```
    if len(coloring) == len(graph):
```

```
        return True
```

```
    region = next(r for r in graph if r not in coloring)
```

```
    for color in colors:
```

```
        if is_safe(region, color):
```

```
            coloring[region] = color
```

```
    if solve():
```

```
        return True
```

```
del coloring[region]
```

```
return False
```

```
if solve():
```

```
    print("Map Coloring Solution:")
```

```
    for region, color in coloring.items():
```

```
        print(region, "->", color)
```

```
else:
```

```
    print("No solution exists")
```

Input

Regions: A, B, C, D

Colors: Red, Green, Blue

Output

```
Map Coloring Solution:  
A -> Red  
B -> Green  
C -> Blue  
D -> Red
```

Result

Thus, the Map Colouring problem was successfully implemented using the Constraint Satisfaction Problem (CSP) technique in Python.

